

08 · Multicore-Synchronisation & Safety

HSEM, Watchdogs und MPU-Grenzen · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 51

Willkommen in der Welt der Robustheit!

- Achter Termin: konkurrierender Speicher, HSEM, Watchdog, MPU
- Dieselbe Plattform wie VL6/VL7: STM32H755, M7 Bare-Metal, M4 FreeRTOS
- Shared Memory, Cache und Boot-HSEM bleiben die Begriffe von VL6
- Das Labor zeigt Mechanismen, es ist keine Automotive-Freigabe
- Kein Cube-Projekt im Repo: Takte, HAL-Symbole und HSEM-IDs dort prüfen

Notizen

Lernziele: Lost Update erklären, ein HSEM-Warten ohne verlorenes Aufwecken skizzieren, ein Watchdog-Fenster aus dem echten Takt rechnen, die MPU-Grenze benennen. Hardwareeigenschaft, Konfiguration, Entwurfsregel und Messung bleiben getrennt.

Folie

2 / 51

Agenda für heute

1. Wann Software reicht und wann nicht
2. HSEM: Besitz, Process-ID, Aufwecken
3. IWDG und WWDG mit Rechnung
4. EWI-Budget und MPU-Grenzen
5. Drei Laborteile, ohne erfundene Messwerte

Notizen

Boot-Schloss und Nutzdaten-Schloss sind zwei IDs. ID 0 bleibt dem Start vorbehalten, wenn das Template sie nutzt.

Recap: Der Ping-Pong Datenaustausch

- Zwei Flags genügen nur mit getrennten Schreibrechten, sichtbaren Zugriffen, Speicherordnung und einem Datenvertrag
- `volatile` ist kein Schloss, kein Cache-Bypass und keine Barriere
- Non-cacheable Region aus VL6 bleibt; DMB bleibt
- HSEM serialisiert den Zähler, es macht den Cache nicht kohärent

Notizen

Payload vor dem Zustandswechsel schreiben, danach erst veröffentlichen. Dieselbe Regel wie VL6, jetzt mit einem Hardware-TryLock drumherum.

Das ungelöste Problem

- Ein FreeRTOS-Mutex gilt für Tasks **dieses** Kernels
- `__disable_irq` maskiert nur den lokalen Kern
- Eine ungeschützte Read-Modify-Write auf einem gemeinsamen Zähler ist nicht atomar
- Geeignete Atomics und ein korrektes Inter-Core-Protokoll können Software-Synchronisation sein
- Ob sie hier tragen, hängt von Region und Kohärenz ab
- Im Labor nehmen wir HSEM, nicht weil Software unmöglich wäre

Notizen

„Völlig nutzlos“ und „kann man nicht in Software lösen“ gelten nicht. Sie gelten für das naive lokale Mutex und für das lokale Interruptverbot.

Warum ein FreeRTOS-Mutex hier versagt

Bedarf	Mittel
M7 gegen M4	HSEM plus Sichtbarkeit aus VL6
zwei M4-Tasks	RTOS-Mutex oder gleichwertig
ISR und Task am selben Schloss	festgelegtes Protokoll, keine blockierende API in der ISR

`HAL_HSEM_FastTake` ist kein Task-Mutex, wenn mehrere Tasks oder eine ISR desselben Kerns dieselbe ID nehmen.

Notizen

FastTake trägt Process-ID 0. Zwei Nutzer mit ID 0 kann der HSEM nicht unterscheiden.

Die Multicore Race Condition

Schematisch, nicht taktgenau. Startwert 5, zwei Inkremente, erwartet 7.

Schritt	M7	M4	RAM
lesen	5		5
lesen		5	5
rechnen	6	6	5
schreiben	6		6
schreiben		6	6

Geschützt, nacheinander: 5 → 6 → **7**.

Notizen

Dieselbe Geschichte wie VL4, nur über die Busmatrix. Die Zeichnung ist kein Trace.

Visualisierung: Multicore-Zerstörung



Notizen

Abbildung A. Keine Nanosekunden. Die zweite Spur setzt voraus, dass der Verlierer nicht mit seinem alten Registerwert schreibt.

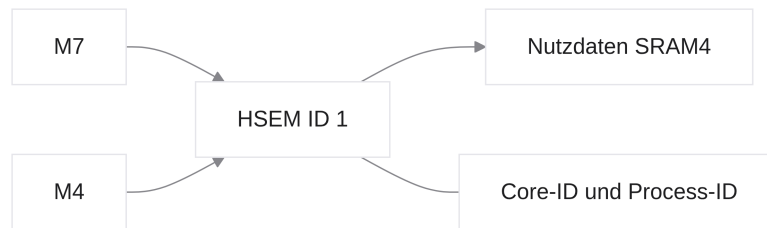
Die Anforderung: Ein Hardware-Schloss

- 32 HSEM-Instanzen auf dieser Familie; im RM0399 der Revision zählen
- Ein Take ist eine atomare Busoperation
- `HAL_ERROR` heißt: Erwerb gescheitert, nicht „Bus beschäftigt“
- Keine Zusage in Nanosekunden und kein garantierter Extra-Takt
- TryLock ist keine Warteschlange und keine Priority Inheritance

Notizen

Wer regulär freigeben darf, steht im RM und im HAL-Treiber der installierten Version. Nicht „jeder Kern“. ReleaseAll ist ein privilegierter, key-basierter Sonderweg.

Einführung in Hardware Semaphores (HSEM)



Frei → belegt durch einen Besitzer → regulär von diesem Besitzer frei.

Recovery steht daneben, nicht auf diesem Pfeil.

Beispiel-ID 1, nur wenn ID 0 der Boot ist und ID 1 frei ist.

Notizen

Abbildung B. Process-ID ist eine Softwarekennung, die wir an `HAL_HSEM_Take` übergeben. Sie ist nicht die FreeRTOS-Task-Handle.

HSEM Architektur im STM32H755

- `HAL_HSEM_Take(id, processId)` schreibt die gewählte Kennung
- `HAL_HSEM_FastTake(id)` nutzt Process-ID 0
- Ob ein zweiter Take desselben Besitzers `HAL_OK` oder `HAL_ERROR` liefert, liest man im Treiber nach
- Das nicht raten und nicht als Rekursion verkaufen

Notizen

Mehrere Tasks einer Process-ID brauchen lokal ein RTOS-Objekt. Die ISR bekommt ein eigenes, nicht blockierendes Protokoll.

Atomarität auf Silizium-Ebene

Der HSEM entscheidet den Erwerb atomar. Er ordnet nicht automatisch Payload-Schreibvorgänge davor und danach.

Reihenfolge im Labor, Region wie VL6:

1. Take erfolgreich
2. nötige Barriere, Payload lesen oder schreiben
3. Barriere vor Release
4. Release

Clean/Invalidate nur, wenn die Region cachebar ist. Die Basis bleibt non-cacheable. DMB bleibt auch dann.

Notizen

Keine Annahme, die HAL habe alle Barrieren schon eingebaut. Den Treiber der Cube-Version lesen.

Die HAL-API für HSEM

```
if (HAL_HSEM_Take(1, 0x37u) == HAL_OK) {  
    __DMB();  
    shared_count += 1u;  
    __DMB();  
    HAL_HSEM_Release(1, 0x37u);  
} else {  
    /* nicht bekommen, nicht anfassen */  
}
```

ProcessID 0x37 ist 8 Bit (0...255), Lehrannahme B, keine Core-ID und keine Task-ID. Der M4 benutzt 0x34 an derselben HSEM-ID.

Notizen

Unter dem Schloss kein `printf` und kein `Delay`. Der Abschnitt bleibt der Zähler.

Flowchart: HSEM Take

Erfolg: Daten, Release. Fehler: nicht warten, als wäre die Benachrichtigung schon der Besitz.

Notizen

TryLock kann sofort scheitern, während der andere Kern legal hält. Das ist kein Busfehler.

Warten auf den HSEM: Polling vs. Interrupts

Falsches Muster: Take scheitert, **danach** IRQ scharf, dann für immer schlafen.

Dazwischen kann der Besitzer freigeben. Die Benachrichtigung geht verloren. Der Task schläft weiter.

Richtig, in dieser Reihenfolge:

1. Benachrichtigung scharf, alte Pending-Lage klären
2. Take erneut
3. Bei Erfolg: Benachrichtigung aus, arbeiten
4. Bei Fehler: auf ein **speicherndes** RTOS-Ereignis warten
5. Nach dem Wecken wieder Take
6. Timeout: Versuch beenden, nicht endlos

Eine Benachrichtigung ist kein Besitznachweis. Danach kann ein Dritter das Schloss schon wieder haben.

Notizen

Abbildung C. Frage: Warum ist die Notification kein Besitz? Weil sie nur sagt, dass jemand freigegeben hat. Der nächste Take kann verlieren. Ob der HAL-Handler die Benachrichtigung ausmacht und sie neu aktiviert werden muss, steht im installierten `stm32h7xx_hal_hsem`.

HSEM Interrupts: M7 weckt M4-Task

Pfad: NVIC → HSEM-IRQ des M4 → `HAL_HSEM_IRQHandler` → Callback → nur ISR-taugliches Signal, zum Beispiel Thread-Flag.

NVIC-Priorität auf der erlaubten Seite der Syscall-Grenze, wie VL5.

Der Task versucht Take erneut. Verliert er, wartet er wieder, mit neu scharf geschalteter Benachrichtigung.

Notizen

Hochpriorities Busy-Wait lässt andere **Tasks** verhungern, nicht automatisch den ganzen Kernel. Ein HSEM ohne Inheritance kann Priority Inversion erzeugen. Der Release-IRQ hebt die Priorität des Besitzers nicht an.

Multicore Deadlock-Gefahr

Zwei IDs in umgekehrter Reihenfolge, gleiche Coffman-Logik wie VL4, jetzt über Kerne.

Lokale RTOS-Locks und HSEM in eine globale Ordnung. Timeout ohne Rollback ist kein Freispruch.

Notizen

Der andere Kern läuft weiter, während eine Task blockiert. Das ist kein Chip-Stillstand.

Die Lösung: HSEM Clear by Core ID

`HAL_HSEM_ReleaseAll` ist der Sonderweg mit Key, nicht die normale Freigabe.

Ein fehlender Heartbeat beweist nicht, dass der andere Kern nicht mehr schreibt. DMA und der zweite Busmaster zählen mit.

Nach einem gewaltsamen Freigeben ist die Struktur vielleicht halb geschrieben. Nicht einfach weiterzählen.

Sicherer Lehrpfad, wenn wir den anderen Kern nicht zuverlässig anhalten können: Gesamt-Reset, Shared-Block neu initialisieren, Handshake von vorn. Kein „Schloss auf, normal weiter“.

Notizen

Abbildung D: Timeout-Verdacht → Ausgangszustand → Reset → Daten neu, nicht weiterverwenden. Key und Core-ID-Konstanten aus dem Header der HAL, nicht aus dem Gedächtnis.

Folie

18/51

Zusammenfassung Hardware-Semaphoren

- HSEM entscheidet den Erwerb, nicht die Sichtbarkeit
- Process-ID selbst wählen, FastTake ist ID 0
- Warten nur mit speicherndem Ereignis und erneutem Take
- ReleaseAll ist Recovery, kein Ping-Pong

Notizen

Abbildung D bleibt die Grenze: ohne Isolation kein Weiterbetrieb auf halben Daten.

Folie

19/51

Was ist „Safety“ in Embedded Systems?

- Reaktionszeit und sicherer Zustand kommen aus der **Anwendung**
- Lock, Watchdog und MPU sind Lehrstücke, kein Sicherheitsnachweis
- „Normen verlangen immer Reset“ streichen
- Ein Bit in `uint16_t`: `0x0064` kann zu `0x8064` werden, nicht zu 65535
- Dagegen helfen Plausibilität, Erkennung und Hardwaremaßnahmen, nicht der Satz „kein C-Code“

Notizen

100 dezimal ist `0x0064`. Ein gesetztes höchstes Bit ergibt 32868, nicht die Umkehrung aller Bits.

Ursachen für System-Crashes

Fehler	Watchdog sieht ihn?
CPU-Hog ohne Refresh	ja, wenn niemand sonst füttert
Deadlock, Idle füttert	oft nein
falsches Ergebnis, Takt stimmt	nein
anderer Kern tot, dieser füttert	nein

Heartbeat, Fortschritt und fachlich richtiges Ergebnis sind drei Dinge.

Notizen

Abbildung F hängt an dieser Tabelle. Ein laufender Supervisor ist kein Allwissender.

Das Konzept des Hardware-Watchdogs

Der STM32H755 hat zwei IWDG- und zwei WWDG-Instanzen. Welche einen Kern, eine Domain oder den Chip zurücksetzt, steht in RCC und im Datenblatt der Revision.

Nicht „immer der ganze Chip“.

Ein Watchdog überwacht eine Frist. Er beweist keine funktionale Sicherheit.

Notizen

Abbildung E: M7/D1, M4/D2, IWDG an LSI, WWDG an ihrem APB. Reset-Pfeil nur so weit zeichnen, wie die Konfiguration reicht.

Die Pflicht der Software (Kicking)

Refresh nur nach einem abgeschlossenen Arbeitsschritt, dessen Nachweis noch frisch ist.

Nicht: „niemals in der ISR“. Sondern: eine ISR, die immer füttert, verdeckt einen Hänger.

Alte Bits nicht wiederverwenden. Den Satz atomar auswerten. Unterschiedliche Perioden brauchen eigene Fristen.

Notizen

Ein Event-Bitset, das man auf einmal löscht, verliert ein Bit, das zwischen Lesen und Löschen gesetzt wurde. Erst tauschen, dann prüfen.

Folie

23/51

Watchdog Typ 1: Independent Watchdog (IWDG)

$$t = (R + 1) \times P / f_{\text{LSI}}$$

Lehrannahme $R = 500$, $P = 32$, $f = 32 \text{ kHz}$:

$$501 \times 32 / 32000 = 0,501 \text{ s}$$

32 kHz ist der Nennwert, kein Quarz. Min und Max aus den LSI-Grenzen des Datenblatts, dann die Refresh-Marge.

Das Fensterregister des IWDG prüfen. „IWDG kann nie zu frühes Füttern sehen“ gilt nicht ungeprüft für diese Familie.

Notizen

Abbildung E, zweite Hälfte: Nominalband und eine Refresh-Marke mit Abstand zu beiden Enden. Die 501 ms sind die Nennrechnung, keine Messung.

Folie

24/51

Der Vorteil des LSI

Der IWDG hängt nicht am HSE. Das heißt nicht: „HSE tot, also steht alles und der Watchdog rettet immer.“

Versorgung, Clock-Security und Fallbacks können den Ausfall anders aussehen lassen. Gemeinsame Hardwarefehler treffen auch den LSI.

Notizen

Unabhängigkeit so weit zeichnen, wie das Datenblatt sie gibt, nicht weiter.

Konfiguration des IWDG

Instanz, LSI an, Prescaler, Reload, Fenster falls genutzt, Start, Debug-Freeze bewusst an oder aus.

Messung mit und ohne Debugger unterscheiden. Freeze im Debug versteckt den Reset.

Notizen

RCC-Resetflags lesen, bevor irgendwer sie löscht.

Die Gefahr im RTOS: Wo füttern wir?

Anti-Pattern: Timer-ISR füttert, die Task hängt.

Idle-Hook: alle betroffenen Tasks blockiert, Idle läuft, füttert, der Fehler bleibt unsichtbar.

Supervisor: nur füttern, wenn **jede** nötige Epoche und der andere Kern frisch sind. Seine eigene Laufzeit begrenzen.

Notizen

Abbildung F: drei Streifen. Blind, Idle, frische Epochen mit ausgelassenem Refresh. Die Tabelle von oben bleibt die Legende.

Anti-Pattern: Füttern im Timer-Interrupt

```
void TIM_IRQ(void) {  
    kick(); /* läuft auch, wenn die Anwendung steht */  
}
```

Notizen

Die ISR beweist, dass der Takt und dieser Handler leben. Nicht, dass der Regler einen Schritt beendet hat.

Pattern 1: Füttern in der Idle-Task

Nur zulässig, wenn „Idle kommt dran“ wirklich die Sicherheitsaussage ist. Für blockierte Worker ist das die falsche Aussage.

Notizen

Idle ist der Beweis, dass der Scheduler einen Tick hatte, nicht dass die Anwendung fortgeschritten ist.

Pattern 2 (Best Practice): Die Watchdog-Task

```
/* Lebenszähler, nicht die Sensorsequenz. Start beide 0, seen =  
uint32_t e4 = epoch_m4; /* je ein lesbarer Wert, kein Paar ohne  
uint32_t e7 = epoch_m7;  
if ((e4 != seen_m4) && (e7 != seen_m7))  
    kick(); /* nur frischer Fortschritt beider Worker  
else  
    fault_no_refresh(); /* Fenster läuft aus, kein Kick */  
seen_m4 = e4;  
seen_m7 = e7;
```

Jeder Worker erhöht seine Epoche einmal pro abgeschlossener Runde, auch wenn kein externes Ereignis kam. Die Sensorsequenz darf dann stehen bleiben. Dieselbe Epoche zweimal ist kein neuer Fortschritt. Die beiden Zähler liegen nicht in einem atomaren Paar: je eine stabile Kopie, Reihenfolge wie in VL6.

Notizen

Der andere Kern braucht einen eigenen, frischen Zähler. Ein lokaler Mutex sieht ihn nicht.

Die Schwäche des IWDG

Ohne Fenster sieht der einfache Timeout einen zu schnellen Unsinn nicht, **wenn** das Fenster aus ist.

Mit Fensterregister kann frühes Füttern je nach Betriebsart doch zählen. Die Betriebsart nennen, nicht nur den Namen IWDG.

Notizen

Vergleich mit WWDG erst, wenn beide Konfigurationen auf dem Tisch liegen.

Typ 2: Window Watchdog (WWDG) Einführung

WWDG1 an APB3, D1. WWDG2 an APB1, D2.

Das M4-Labor nimmt die Instanz, deren PCLK wir kennen. CPU-Takt ist nicht dieser PCLK.

$$t_{\text{tick}} = 4096 \times \text{Prescaler} / f_{\text{PCLK}}$$

Lehrannahme B, innerhalb des APB-Maximums: WWDG2, $f_{\text{PCLK}} = 100 \text{ MHz}$, Prescaler 8. Zum Vergleich 50 MHz, nicht 200 MHz.

$$t_{\text{tick}}(100 \text{ MHz}) = 4096 \times 8 / 10^8 = 0,32768 \text{ ms}$$

Bei 50 MHz ist der Tick doppelt so lang, 0,65536 ms. Das APB-Maximum hängt von VOS ab und liegt laut Datenblatt bei bis zu 120 MHz. 200 MHz ist kein zulässiger PCLK.

Notizen

Abbildung G braucht die Projekt-PCLK. Solange sie fehlt, bleiben die ms-Achsen als Illustration gekennzeichnet. Tickquantisierung und Jitter extra addieren. Testzeiten mit Abstand zu beiden Grenzen.

Das Fenster-Konzept (Window Watchdog)

Lehrwerte: Startzähler $T = 127$, Fenster $W = 80$. EWI bei 0×40 . Reset beim Übergang $0 \times 40 \rightarrow 0 \times 3F$.

Lehrregel: Refresh ist zu früh, solange der Zähler **größer** als 80 ist. Gleichheit 80 gilt in diesem Beispiel als erlaubt. Den Vergleich im RM0399 gegenlesen; die Bitformel dieser Revision ist hier nicht zeilenweise geprüft.

Bei 0,32768 ms je Tick: 47 Schritte von 127 auf 80 dauern 15,4 ms (noch zu früh). 16 weitere Schritte von 80 auf 0×40 dauern 5,2 ms (Fenster). Danach ein Tick bis zum Reset.

- 5 ms nach dem Reload: Zähler noch über 80, zu früh, Reset ohne EWI.
- 18 ms: Zähler etwa 72, im Fenster, zulässig.
- kein Refresh über 25 ms: Reset nach $0 \times 3F$.

`osDelay` rastet auf 1 ms und beweist das Fenster nicht. Marker an Reload und Refresh, Zeitbasis der Lehrkonfiguration.

Ein ausgelassener Programmschritt löst nicht automatisch einen frühen Refresh aus, wenn eine feste Wartezeit danach immer noch im Fenster liegt.

Notizen

Prescaler 8, Window 80, Reload 120 sind Zählerwerte. Die Millisekunden kommen erst aus f_{PCLK} .

Visualisierung des WWDG-Fensters

Drei Verläufe auf derselben Achse, in Zählerschritten und, sobald PCLK bekannt ist, in ms:

- Refresh zu früh $\rightarrow$ Reset ohne auf EWI zu warten
- Refresh im Fenster $\rightarrow$ Zähler neu, kein Reset
- kein Refresh $\rightarrow$ EWI bei 0×40 , Reset beim nächsten Tick nach $0 \times 3F$

Notizen

Abbildung G. Gleicher Maßstab, drei Marker. Keine Behauptung, 15 ms sei bei jedem APB gültig.

IWDG und WWDG im Doppelpack

IWDG: lange Frist, eigener LSI, Fenster nur wenn konfiguriert.

WWDG: kurzes Fenster am APB, frühes und spätes Füttern, EWI möglich.

Zusammen nur, wenn beide Reset-Wirkungen verstanden sind. Sonst erklärt ein Reset den falschen Hund.

Notizen

Reset-Cause-Flags sichern, bevor sie gelöscht werden. Debug-Freeze für jede Instanz einzeln.

Warum WWDG? (Fehlerhafte Ausführung)

Eine Schleife, die zu oft füttert, kann am Fenster scheitern, am reinen Timeout nicht.

Eine Schleife, die in der richtigen Pause füttert, besteht das Fenster und kann fachlich falsch sein. Der Hund sieht die Pause, nicht das Ergebnis.

Notizen

Das ist der Sinn der Tabelle aus dem Crash-Block.

Das Sicherheitsnetz: EWI

EWI ist ein gewöhnlicher, maskierbarer IRQ. Kein NMI. Keine Rettung, wenn die IRQ aus ist, der Kern steht oder der Handler selbst hängt.

Zu früher Refresh kann resetten, **ohne** dass EWI kam.

Bei 0×40 kann ein erlaubtes Refresh den Reset noch verhindern. „Wir füttern im Fehlerfall nicht“ ist eine Entscheidung, keine Hardware-Unmöglichkeit.

Budget: ein WWDG-Tick, abzüglich Eintrittslatenz und Handlerzeit. Kein pauschales „wenige Mikrosekunden“. Kein Flash-Programmieren in diesem Rest.

Ein RAM-Satz braucht Retention und Cache-Regeln. Sonst sagen wir, dass er den Reset nicht überlebt.

Notizen

Abbildung H. Pfad: NVIC → IRQ der richtigen Instanz → `HAL_WWDG_IRQHandler` → Callback.

Höchste IRQ-Priorität: keine RTOS-API. Die Labor-LED zeigt nur, dass der Callback lief. Sichere Pegel schon während Reset und Boot, nicht erst im Callback. „PWM auf null“ ist ein Lehrbeispiel, keine universelle Reaktion.

Folie

37 / 51

Nutzen des EWI (Safe Shutdown)

Ursache merken, definierte Ausgänge, optional kurzer RAM-Eintrag, dann die gewählte Policy: Refresh vermeiden und resetten lassen, oder bewusst noch einmal füttern.
Die Policy aufschreiben. Beides gleichzeitig behaupten geht nicht.

Notizen

Der Callback ist kein zweites Betriebssystem.

Folie

38 / 51

HAL-API für den WWDG

Clock an, Init mit Prescaler, Fenster, Reload, EWI an, NVIC, Handler, Callback.
`osDelay` nur in der Task und nur mit bekanntem Tick und bekanntem PCLK, sonst keine früh/spät-Etiketten.
GPIO-Marker für das Fenster. Eine LED sieht einen kurzen EWI-Puls nicht zuverlässig.

Notizen

Ohne Oszilloskop die Zeiten als offen markieren, nicht als gemessen.

Folie

39 / 51

Zusammenfassung: Watchdogs

- IWDG-Nennwert 501 ms nur bei 32 kHz, Reload 500, Prescaler 32
- WWDG-Tick skaliert mit PCLK
- Füttern ist ein Fortschrittsnachweis
- EWI ist ein enges, maskierbares Fenster vor dem späten Reset

Notizen

Frage: Warum übersieht der Idle-Hook einen Deadlock? Weil Idle weiterläuft, obwohl die Worker blockiert sind.

Speicherschutz: Die MPU (Erinnerung)

Die MPU prüft Regionen des **konfigurierten** Kerns. Keine C-Arraygrenzen.

Ein Zugriff hinter dem Array bleibt unsichtbar, wenn er in einer erlaubten Region liegt.

DMA und der andere Kern laufen daran vorbei. Cache-Attribute aus VL6 sind keine Task-Isolation.

Notizen

Abbildung I: erlaubte Region, verbotene Region, Array innen. Drei Fälle: Treffer im Array, Überlauf ohne Regionsfehler, Zugriff in die verbotene Region.

Der primäre Zweck der MPU

Ein verbotener Zugriff führt über MemManage, wenn die Fault-Maske das erlaubt, sonst oft zum HardFault.

Das ist kein Halt „in derselben Nanosekunde“ und kein sicherer Stillstand. Das RTOS läuft danach nicht automatisch gesund weiter.

Notizen

Exception-Stacking auf einem schon zerstörten Stack kann den Fault selbst verderben.

Use-Case 1: Null-Pointer-Protection

Ein uninitialisierter Zeiger ist nicht automatisch NULL.

Adresse 0 auf dem H755 kann eine ITCM- oder Boot-Abbildung sein. Das Datenblatt und das RM prüfen, bevor jemand „NULL ist frei“ sagt.

*NULL in C ist undefiniertes Verhalten. Ein reproduzierbarer Test schreibt eine dokumentierte Adresse in einer Region, die die MPU verbietet, und kontrolliert das Disassembly.

Überlappende Regionen: die höhere Regionsnummer gewinnt. Eine erlaubende Region darüber schluckt den Test.

Notizen

Frage: Warum sieht die MPU nicht jeden Arrayüberlauf? Weil sie Regionen prüft, nicht die Länge des C-Arrays. Der Überlauf kann in der erlaubten Region bleiben.

Use-Case 2: Stack-Overflow Erkennung

Der Stack wächst nach unten. Die Guard-Region liegt an der **unteren** Grenze, ausgerichtet, mit den Rechten der konkreten MPU.

Ein Sprung über den Guard oder ein Überlauf, der in einer erlaubten Nachbarregion landet, bleibt unsichtbar.

Der gewöhnliche FreeRTOS-CM4-Port legt nicht für jede Task eine Guard an. Dafür braucht es den MPU-Port und seine Konfiguration.

Notizen

Abbildung J: SP, Wachstumsrichtung, Guard, Nachbar. Matrix: Task, Handler, DMA und anderer Kern. Nur die Task-Spalte sieht die Guard.

Privileged vs. User Mode

Thread Mode und Handler Mode sind nicht dasselbe wie privilegiert und unprivilegiert.

Background-Region, Wrapper und was ein Task-Fault auslöst, gehören in die Port-Doku.

„Nur die Task stirbt, der Rest läuft sicher“ nur, wenn dieses Recovery wirklich gebaut und geprüft ist. Hier ist es das nicht.

Notizen

Handler-Mode bleibt privilegiert, auch wenn Tasks unprivilegiert laufen. DMA hat keine MPU dieses Kerns.

Zusammenfassung MPU & Safety

- Regionen, nicht Arraylängen
- ein Kern, nicht der Chip
- Fault ist eine Ausnahme, kein sicherer Zustand
- Guard nur so weit, wie der Port sie wirklich anlegt

Notizen

Die drei Fragen: Notification ist kein Besitz. Idle übersieht blockierte Tasks. Die MPU übersieht Überläufe in erlaubten Regionen.

Laborvorbereitung: Multicore-Safety

Drei Teile, nicht ein Brei.

Punkt	Wert
Board	nur wenn NUCLEO-H755ZI-Q
HSEM Boot	oft 0, nicht anfassen
HSEM Zähler	1, wenn frei
Shared	SRAM4 wie VL6, non-cacheable
WWDG	Instanz des M4, PCLK eintragen
IWDG	Instanz und LSI-Grenzen eintragen

Zähler einmal initialisieren, bevor beide loslegen. Je Kern N Inkremente. Erwartung $2N$. Ein ungeschützter Lauf, der zufällig $2N$ trifft, beweist keine Freiheit von Races.

Notizen

Abbildung K: Boot-HSEM getrennt vom Zähler-HSEM, Messpins getrennt von der LED. Abbildung L: Erwartung und Beobachtung in zwei Spalten. Leere Beobachtung ist ehrlich.

Schritt 2: Der asynchrone Multicore-Zugriff

M7 zählt nur erfolgreiche Inkremente. Timeout nach 2 s lokalen HAL_GetTick macht den Lauf ungültig, nicht zu einem Sollwert $2N$.

```
uint32_t hits = 0, t0 = HAL_GetTick();
while (hits < N) {
    if ((uint32_t)(HAL_GetTick() - t0) > 2000u) { fail(); return; }
    if (HAL_HSEM_Take(1, 0x37u) != HAL_OK) continue;
    __DMB();
    shared_count += 1u;
    __DMB();
    HAL_HSEM_Release(1, 0x37u);
    hits++;
}
```

Der M4 tut dasselbe mit ProcessID 0x34 und osKernelGetTickCount, und gibt danach die CPU mit osDelay(1) ab. Der Zähler startet einmal bei 0. Gelesen wird er erst, wenn beide hits == N gemeldet haben. HSEM stellt keine Cache-Kohärenz her; die Region bleibt non-cacheable.

Warten ohne Lost Wakeup ist ein eigener Versuch, nicht diese Zählschleife. Mehrere Tasks an ID 1 brauchen zusätzlich einen lokalen Mutex.

Notizen

N endlich. Danach beide fertig und dann erst $2N$ lesen. Sonst liest man einen Zwischenstand.

Schritt 3: Den WWDG integrieren

Clock, Init, Fenster, Reload, Prescaler, EWI, NVIC, richtiger IRQ-Handler, Callback.
Erst die Tabelle aus f_{PCLK} füllen, dann Delays wählen, mit Abstand zu früh und spät.
Debug-Freeze dokumentieren. Resetflags vorher sichern.

Notizen

15 ms, 50 ms und 5 ms aus der alten Anleitung nicht übernehmen, solange der Tick 0,33 ms oder 0,16 ms sein kann.

Schritt 4: Den Wachhund füttern & Crashen

Ein Experiment nach dem anderen: früher Refresh, gültiger Refresh, kein Refresh, CPU-Hog, blockierte Tasks mit Idle-Füttern.

Sonst verdeckt ein Reset den nächsten Fall.

Marker-GPIO. LED nur als grobes Lebenszeichen.

Notizen

Erwartung analytisch. Beobachtung erst nach dem Lauf. Hier bleibt sie offen.

Schritt 5: Den Safe-Shutdown implementieren

Callback: Ursache merken, Ausgänge auf die definierten Reset-Pegel, kein Flash, kein RTOS, wenn die Priorität das verbietet.

Policy aufschreiben. Die LED beweist den Aufruf, nicht den sicheren Zustand der Anwendung.

MPU-Test: eine verbotene, dokumentierte Adresse, Disassembly prüfen. Kein *NULL als Beweis.

Notizen

Fault-Eskalation notieren: MemManage oder HardFault. Kein „Task tot, System gesund“, solange der Port das nicht tut.

Die Essenz des Tages

- Lokales Mutex und lokales IRQ-Verbot sind keine Inter-Core-Schlösser
- HSEM plus VL6-Sichtbarkeit; Notification ist kein Besitz
- 501 ms IWDG nur unter der Nennannahme; WWDG-ms nur mit PCLK
- MPU sieht Regionen eines Kerns, nicht jede C-Grenze
- Das Labor demonstriert. Es gibt nichts frei

Notizen

Lösungen: Nach dem Aufwecken erneut Take, weil jemand anderes schneller war. Idle läuft bei blockierten Tasks weiter. Ein Überlauf in einer erlaubten Region erzeugt keinen Fault. Offene Punkte: HAL-Revision, freie HSEM-ID, PCLK, LSI-Grenzen, Reset-Scope, ob ReleaseAll im Lehrprojekt überhaupt sicher ist.

Kein Hardwarelauf in diesem Repo.